

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5

9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I MONIKA.B(192571002) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — MOCK INTERVIEW ANSWERS

Q1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Query processing is the sequence of steps a DBMS follows to translate a high-level SQL query into an efficient, executable low-level plan that retrieves the required data. It consists of four main phases:

- **Parsing and Translation:** The SQL query is checked for syntax and semantic correctness, and then translated into an internal representation such as a relational algebra expression or a parse tree.
- **Query Optimization:** The optimizer generates multiple equivalent execution plans for the same query and estimates the cost of each, using statistics like table size, indexes, and selectivity, to choose the most efficient one.
- **Query Code Generation / Evaluation Plan:** The chosen plan is converted into an evaluation plan, essentially an annotated relational algebra tree specifying the exact algorithm to use for each operation, such as a sort-merge join or a hash join.
- **Execution:** The query evaluation engine executes the plan against the database and returns the result set to the user or application.

The optimizer decides the best plan by first enumerating alternative access paths, for example, a full table scan versus an index scan, and alternative join orders and join algorithms. It then estimates the cost of each candidate plan in terms of disk I/O, CPU usage, and memory, using catalog statistics such as the number of tuples, distinct values, and existing indexes. Techniques such as heuristic transformation rules and cost-based dynamic programming are applied to prune the search space, and the plan with the lowest estimated cost is selected for execution. This ensures the query returns correct results while minimizing resource consumption and response time.

Q2. What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Both are strategies used by the query optimizer to improve performance, but they differ in approach:

Heuristic (Rule-Based) Optimization

This approach applies a fixed set of transformation rules to the query tree without evaluating the actual cost of the resulting plans. Common heuristics include performing selection and projection operations as early as possible to reduce the size of intermediate relations, combining cartesian products with subsequent selections into joins, and reordering joins so that the most restrictive operations execute first. It is fast because it does not require cost estimation, but it may not always produce the truly optimal plan since it ignores real data statistics.

Cost-Based Optimization

This approach generates several candidate execution plans and estimates the cost of each using statistics stored in the system catalog, such as table cardinality, index availability, and data distribution. It then selects the plan with the lowest estimated cost in terms of CPU, I/O, and memory usage. This method is more accurate but computationally more expensive, since it must evaluate many possible plans, especially for multi-way joins.

When to use each

Heuristic optimization is preferable for simple queries or systems with limited statistics, where speed of optimization itself matters more than finding the absolute best plan. Cost-based optimization is preferred for complex queries involving multiple joins and large tables, where accurate statistics are available and the cost of poor plan selection, such as a full table scan on a huge table, would be far more expensive than the extra time spent optimizing. In practice, modern DBMSs like Oracle and PostgreSQL use heuristics to prune obviously bad plans first and then apply cost-based analysis on the remaining candidates, combining the speed of one with the accuracy of the other.

Q3. How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

:

In a real project, I would implement database connectivity in layers to keep the application maintainable and secure:

- Choose a connectivity API appropriate to the technology stack, for example JDBC for a Java application or ODBC/ADO.NET for a .NET or C++ application.
- Load and register the database driver, then establish a connection using a connection string that specifies the host, port, database name, and credentials, ideally read from a secure configuration file or environment variable rather than hard-coded.
- Use a connection pool, such as HikariCP in Java, so connections are reused efficiently instead of opening and closing a new connection for every query, which improves performance under load.
- Execute queries through prepared statements rather than string concatenation, both for performance and to prevent SQL injection.
- Handle exceptions, commit or rollback transactions explicitly where needed, and close connections/result sets in a finally block or using try-with-resources to avoid resource leaks.

JDBC vs ODBC

JDBC (Java Database Connectivity) is a Java-specific API that allows Java applications to interact with databases through a set of standard interfaces and JDBC drivers written in Java, making it platform-independent since it runs on the JVM. ODBC (Open Database Connectivity) is a language-independent, C-based standard that provides a common interface for any application, in any programming language, to access any ODBC-compliant database through native drivers, but it is tied to the operating system and often requires the ODBC driver manager to be installed.

In short, JDBC is designed exclusively for Java and communicates directly with the database using Java drivers, giving better performance and type-safety within Java applications, while ODBC is broader in scope, usable from many languages such as C, C++, and VB, but generally requires an additional translation layer when used from Java (via the JDBC-ODBC bridge, which is now deprecated). For a modern Java project I would use JDBC directly; for a legacy or multi-language enterprise environment, ODBC offers wider compatibility.

Q4. What are ACID properties? Give a real-world example where each property is critical.

ACID is a set of four properties that guarantee reliable processing of database transactions:

- **Atomicity:** A transaction is treated as a single indivisible unit, either all its operations complete or none do. Example: In a bank fund transfer, if money is debited from Account A but the credit to Account B fails due to a system crash, atomicity ensures the entire transaction is rolled back so the debit is undone and no money is lost.
- **Consistency:** A transaction must take the database from one valid state to another, preserving all defined rules and constraints. Example: If a database enforces that an account balance cannot go negative, consistency ensures that a withdrawal transaction violating this rule is rejected, keeping the database in a valid state.
- **Isolation:** Concurrent transactions must not interfere with each other, as if they were executed serially. Example: If two customers simultaneously book the last available seat on a flight, isolation ensures only one booking succeeds and the other is informed the seat is no longer available, preventing double-booking.
- **Durability:** Once a transaction is committed, its changes must persist even in the event of a system crash or power failure. Example: After an e-commerce order is confirmed and payment is deducted, durability guarantees that even if the server crashes immediately afterward, the order and payment record remain saved when the system restarts.

Together, these properties ensure that a database remains accurate, reliable, and trustworthy even under concurrent access and system failures, which is why they are foundational to transaction management in any production DBMS.

Q5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

When multiple transactions execute concurrently without proper control, several anomalies can occur:

- **Dirty Read:** A transaction reads data that has been modified by another transaction that has not yet committed. If the second transaction later rolls back, the first transaction has used invalid, uncommitted data. Example: T1 updates a product's stock but has not committed; T2 reads the updated stock and proceeds with a sale, but then T1 rolls back, leaving T2's decision based on data that never truly existed.
- **Lost Update:** Two transactions read the same data item and then both update it based on the value they read, causing one update to overwrite and effectively erase the other. Example: T1 and T2 both read a bank balance of 1000, T1 adds 100 and writes 1100, then T2 adds 200 to its own copy (1000) and writes 1200, silently losing T1's update.
- **Unrepeatable Read:** A transaction reads the same data item twice and gets different values because another committed transaction modified it in between. Example: T1 reads a balance as 500, then T2 updates and commits it to 700, and when T1 reads it again within the same transaction it sees 700, an inconsistency within a single logical unit of work.

Two-Phase Locking (2PL) solves these problems by requiring every transaction to acquire all the locks it needs before releasing any lock, dividing the transaction's life into a growing phase, where locks are only acquired, and a shrinking phase, where locks are only released. By holding shared locks on data it reads and exclusive locks on data it writes until the appropriate phase, 2PL prevents another transaction from reading uncommitted data (avoiding dirty reads), prevents two transactions from writing to the same item without seeing each other's changes (avoiding lost updates), and prevents a value from changing between two reads within the same transaction (avoiding unrepeatable reads). 2PL guarantees that the resulting schedule is conflict-serializable, which is the formal basis for correctness in concurrent execution.

Q6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

A deadlock occurs when two or more transactions are each waiting for a lock held by the other, forming a cycle of dependencies so that none of them can ever proceed. For example, T1 holds a lock on item A and waits for a lock on item B, while T2 holds a lock on item B and waits for a lock on item A; both transactions are blocked indefinitely.

Deadlock Detection

The DBMS periodically constructs a Wait-For Graph (WFG), where each transaction is a node and a directed edge from T_i to T_j means T_i is waiting for a lock held by T_j . The system runs a cycle-detection algorithm, essentially a graph traversal such as depth-first search, on this graph at regular intervals or whenever a transaction is forced to wait. If a cycle is found, a deadlock exists, and the DBMS resolves it by selecting a victim transaction, typically the one with the least work done or lowest priority, and rolling it back to break the cycle, then restarting it later.

Deadlock Prevention Strategies

- **Wait-Die scheme:** If an older transaction requests a lock held by a younger transaction, the older transaction is allowed to wait; if a younger transaction requests a lock held by an older one, the younger transaction is aborted (dies) and restarted later. This ensures no cycles can form because waits always go in one timestamp direction.

- Wound-Wait scheme: If an older transaction requests a lock held by a younger one, it wounds (forces abort of) the younger transaction; if a younger transaction requests a lock held by an older one, it waits. This is the preemptive counterpart to Wait-Die.
- Lock ordering / resource ordering: All transactions are required to acquire locks on data items in a predefined global order, which structurally prevents circular wait conditions from ever forming.
- Timeout-based prevention: A transaction that waits longer than a predefined timeout is assumed to be deadlocked and is automatically rolled back, a simpler but less precise alternative to graph-based detection.

Q7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Deferred Update (NO-UNDO/REDO)

In this technique, all updates made by a transaction are recorded only in the log and applied to the actual database only after the transaction commits. Since the database is never modified by an uncommitted transaction, recovery never needs to undo any changes; if a crash occurs before commit, the transaction's log records are simply ignored (as if it never happened). During recovery, the system only needs to redo the operations of committed transactions whose changes might not yet have been flushed to disk.

Immediate Update (UNDO/REDO)

Here, updates are applied to the database as soon as they occur, even before the transaction commits, provided the corresponding log record (with both old and new values) has already been written to stable storage ahead of time, following the Write-Ahead Logging (WAL) protocol. Because uncommitted changes may already be on disk, recovery must be able to undo the operations of transactions that were active but did not commit at the time of the crash, and redo the operations of transactions that committed but whose changes may not have reached disk.

When each is preferred

Deferred update is preferred in systems where transactions are typically short and memory is sufficient to hold all changes until commit, since it simplifies recovery logic by eliminating undo operations entirely and reduces the risk of writing inconsistent data to disk. Immediate update is preferred in systems with long-running transactions or limited buffer memory, where it is impractical to delay all writes until commit; it allows the buffer manager to flush dirty pages to disk earlier for efficiency, at the cost of needing a more complex recovery procedure that supports both undo and redo.

Q8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Shadow Paging is a recovery technique that avoids maintaining logs altogether by using two page tables: the current page table, which points to the most recent versions of database pages that the active transaction is modifying, and the shadow page table, which is a copy of the current page table taken at the start of the transaction and points to the unmodified, consistent pages. When a transaction updates a page, the DBMS writes the modified data to a new physical location on disk rather than overwriting the original page, and the current page table is updated to point to this new page, while the shadow page table continues to point to the original, unaltered page. If the transaction commits successfully, the shadow page table is simply replaced by the current page table (an atomic pointer swap), making all changes permanent at once. If the transaction fails or the system crashes before commit, the system discards the current page table and reverts to the shadow page table, effectively undoing all changes instantly, since the original pages were never touched.

Advantages over log-based recovery

- No need for undo/redo operations during recovery, since committing is just switching a table pointer, making recovery extremely fast.
- No overhead of maintaining and writing log records for every update, simplifying the implementation.

Limitations compared to log-based recovery

- Data fragmentation: because updated pages are written to new locations rather than in place, related pages become scattered across disk over time, degrading locality and performance.
- Garbage collection overhead: old pages that are no longer referenced after a commit must be identified and reclaimed, adding extra processing.
- Difficult to support concurrent transactions efficiently, since maintaining separate shadow copies for multiple simultaneous transactions is complex, whereas log-based methods integrate naturally with concurrency control.
- Not well suited for very large databases or systems requiring fine-grained, high-frequency transactions, which is why almost all modern commercial DBMSs use log-based (WAL) recovery instead of shadow paging.

Q9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

A serializable schedule is a schedule of concurrently executing transactions whose final effect on the database is equivalent to some serial (one-after-another, non-interleaved) execution of the same transactions. Serializability is the primary correctness criterion for concurrent transaction execution, since a serial schedule is always assumed to be correct, and any interleaved schedule that produces the same result is considered correct as well.

Testing Conflict Serializability using a Precedence Graph

A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting operations. This is tested by constructing a precedence graph (also called a serialization graph):

- Create one node for each transaction in the schedule.
- For every pair of conflicting operations, meaning operations from different transactions on the same data item where at least one is a write, add a directed edge from T_i to T_j if an operation of T_i precedes and conflicts with an operation of T_j (for example, T_i writes an item before T_j reads or writes the same item).
- After considering all conflicting operation pairs in the schedule, examine the resulting graph for cycles.
- If the precedence graph contains no cycles, the schedule is conflict-serializable, and a valid serial order can be obtained by performing a topological sort of the graph. If the graph contains a cycle, the schedule is not conflict-serializable, because it would be impossible to reorder the transactions serially without changing the outcome.

This precedence graph technique gives the DBMS, or a database engineer verifying correctness, a systematic and efficient way to check serializability without having to manually enumerate all possible serial orderings of the transactions.

Q10. Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

- Basic 2PL: The transaction is divided into a growing phase, in which it can acquire locks but cannot release any, and a shrinking phase, in which it can release locks but cannot acquire any new ones. Basic 2PL guarantees conflict-serializability, but because locks can be released before the transaction commits, it does not prevent cascading rollbacks; if a transaction releases a lock and another transaction reads that data, then the first transaction later aborts, the second transaction must also be rolled back.
- Strict 2PL: This adds the restriction that all exclusive (write) locks held by a transaction must be held until the transaction commits or aborts, only after which they are released. This prevents other transactions from reading or writing data that has been modified by an uncommitted transaction, thereby avoiding cascading rollbacks, since no other transaction can see the changes until they are final.
- Rigorous 2PL: This is the strictest variant, requiring both shared (read) and exclusive (write) locks to be held until the transaction commits or aborts, not just exclusive locks. This means no other transaction can read or write an item a transaction has touched, in any way, until it is fully complete, which simplifies

concurrency control logic even further and makes the commit order of transactions the same as their serialization order.

Strict 2PL is the most commonly used variant in real-world database systems, including most commercial DBMSs, because it strikes the best balance between correctness and performance: it guarantees conflict-serializability and, critically, avoids cascading rollbacks (a major practical concern, since a single abort could otherwise trigger a chain of rollbacks across many transactions), while still allowing read locks to be released a little earlier than in Rigorous 2PL, offering somewhat better concurrency. Rigorous 2PL is used in some systems where an even simpler, more predictable transaction ordering is desired, but it can reduce concurrency further, and Basic 2PL is mostly of theoretical interest because the risk of cascading rollbacks makes it unattractive for production systems.
